Course Syllabus for CSE-431

1. Course Code: CSE-431

2. Course Title: Compiler Design

3. Course Type: Core Course

4. Level/ Term: Level: 4 Term: II

5. Academic Session: 2020-21

6. Course Teacher: Dr. Mohammed Moshiul Hoque, Professor, Dept. of CSE, CUET
Ashim Dey, Assistant Professor, Dept. of CSE, CUET

7. Prerequisite(s): None

8. Credits: 3

9. Contact Hours: 3 Lectures of 50 minutes per week

10. Total Marks: 300

11. Rational of the Course:

This course comprises the basic architecture of a compiler and the detail about how compiler works in different phases to compile a programming language. The course consists of the basic and detail of lexical, syntax and semantic analysis by a compiler. This course is important to understand and design compiler functions as without compiler any programming language is meaningless. This is a required course for all the students enrolling B. Sc. Engg. in CSE program. The catalogue description of the course is

Course Content:

Basic issues, compiler structure, front end, back end; Lexical analysis: Tokens, patterns, and lexemes, input buffering, transition diagrams, lexical-analyzer generator; Syntax analysis: Elimination of left recursion, left factoring, FIRST and FOLLOW, LL(1) grammars, non-recursive predictive parsing, parser generators; Syntax-directed translation: Syntax directed definitions, inherited and synthesized attributes, dependency graphs, syntax-directed translation schemes; Semantic analysis: Type expressions, type equivalence, type-checking; Run-time environments: Storage organization, static versus dynamic storage allocation, activation trees, activation records; Intermediate code generation: Directed acyclic graphs for expressions, three-address code, quadruples, triples, static single- assignment form; Code generation; Code optimization: Basic blocks and flow graphs, next-use information, optimization of basic blocks.

12. Course Objectives:

a) To introduce the principles and techniques of compiler construction.
b) To develop the concept of different phases of compiler design: lexical

analysis, parsing, semantic analysis, code generation and optimization.

c) To elaborate theoretical and practical details of a modern compiler.
d) To make a student enable building up a full functioning compiler for a high level programming language.

13. Course Learning Outcomes (CLOs) and Mapping of CLOs with Program Learning Outcomes (PLOs)

a) CLOs

	Course Learning Outcomes (CLOs)	Blooms Level (Optional)
CLO1	Identify how a compiler convert a source program into equivalent target program	C1
CLO2	Apply knowledge to build up various components of compiler like lexical analyzer, syntax and semantic analyzer	C3
CLO3	Investigate the existing compilers' working procedure then add features in those as well as build up new concepts	C4

b) Mapping of CLO with PLO

No.	(CLOs)	PLO1	PLO2	PLO3	PLO4	PLO5	PLO6	PLO7	PLO8	PLO9	PLO10	PLO11	PLO12
1	CLO1	X											
2	CLO2			X									
3	CLO3				X								

Part B

14. Course plan specifying content, CLOs, co-curricular activities (if any), teaching learning and assessment strategy mapped with CLOs

`	Topic	Teaching-Learning Methodology	Assessment Method	Corresponding CLOs
Lesson-01	Introduction, functions of compiler, history and application of compiler	Multimedia presentation	Discussion, Q/A	**CLO-1**
Lesson-02	Interpreter & compiler, phases of compilation, symbol table, compiler construction tools	Multimedia presentation, Whiteboard	Discussion, Q/A	**CLO-1**
Lesson-03	Simple syntax directed translation: front & back end of compiler, definition of syntax, grammar & derivation	Multimedia presentation, Whiteboard	Quiz	**CLO-1**
Lesson-04	Simple syntax directed translation: Parse tree, associativity & precedence of operator, postfix notation	Multimedia presentation, Whiteboard	Assignment	**CLO-1**
Lesson-05	Simple syntax directed translation: top-down and bottom-up parsing, left/right recursion, concrete and abstract syntax trees	Multimedia presentation, Whiteboard	Illustration by examples, test exam	**CLO-1**

Lesson-06	Lexical analysis: role of lexical analyzer, token, lexemes, patterns, regular expressions, regular definition	Multimedia presentation, Whiteboard	Problem solving, quiz	**CLO-1**
Lesson-07	Lexical analysis: Finite automata-design of DFAs & NFAs, subset construction algorithm	Multimedia presentation, Whiteboard	Problem solving, assignment	**CLO-1** **CLO-3**
Lesson-08	Lexical Analysis: conversion of regular expression to NFA, practice examples	Multimedia presentation, Whiteboard	Problem solving, assignment	**CLO-1** **CLO-3**
Lesson-09	Lexical Analysis: conversion of regular expression to DFA, practice examples	Multimedia presentation, Whiteboard	Problem solving, assignment	**CLO-1** **CLO-3**
Lesson-10	Syntax Analysis: role of the parser, error recovery strategies, Context-free grammars	Multimedia presentation, Whiteboard	Group discussion, quiz	**CLO-1**
Lesson-11	Syntax Analysis: lexical vs. syntactic analysis, derivations- left most & right most, practice example	Multimedia presentation, Whiteboard	Problem solving, assignments	**CLO-1** **CLO-3**
Lesson-12	Syntax Analysis: ambiguity, eliminating ambiguity, recursion, left factoring, practice examples	Multimedia presentation, Whiteboard	Problem solving, assignments	**CLO-1**

Lesson-13	Syntax Analysis: top-down parsing with example, computing FIRST and FOLLOW	Multimedia presentation, Whiteboard	Quiz, discussion	**CLO-1**
Lesson-14 &15	Syntax Analysis: recursive decent parsing, LL1(1) grammars, predictive parsing, bottom-up parsing, shift reduce, LR	Multimedia presentation, Whiteboard	Quiz, problem solving	**CLO-1**
Lesson-16	Syntax Analysis: SLR and LALR parsing	Multimedia presentation, Whiteboard	Problem solving, surprise test	**CLO-1**
Lesson-17 & 18	Semantic Analysis: importance, syntax directed definition (SDD), inherited vs. synthesized attributes, annotated parse tree, semantic rules, dependency graph	Multimedia presentation, Whiteboard	Illustration by example, assignment, discussion	**CLO-1**
Lesson-19	Semantic Analysis: evaluative order of attributes, S-attribute, L-attribute, syntax directed translation (SDT)	Multimedia presentation, Whiteboard	Problem solving, quiz	**CLO-1**
Lesson-20	Semantic Analysis: syntax tree, postfix translation, SDT implementation technique	Multimedia presentation, Whiteboard	Problem solving, discussion	**CLO-1**

Lesson-21	Type checking: type checking, static vs. dynamic checking, function of type checker, type expressions, type equivalence	Multimedia presentation, Whiteboard	Problem solving, assignment	**CLO-1**
Lesson-22	Type checking: operations within expression, type checking rules, type conversions, semantic rules for conversion	Multimedia presentation, Whiteboard	Discussion, quiz	**CLO-1**
Lesson-23	Type checking: implicit and explicit conversion, type conversion into expression	Multimedia presentation, Whiteboard	Illustration by examples, test	**CLO-1**
Lesson-24 & 25	Review of previous lectures, Intermediate code generation: different type of IC representation, DAG basics, DAG for expression, DAG implementation	Multimedia presentation, Whiteboard	Illustration by examples, assignment, test	**CLO-1**
Lesson-26	Intermediate code generation: DAG vs. syntax tree and DAG vs. parse tree, value number method for constructing DAG, hash function implementation	Multimedia presentation, Whiteboard	Illustration by examples, quiz	**CLO-1**

Lesson-27	Three address code: basic definition, application, properties, format, program to three address code example	Multimedia presentation, Whiteboard	Discussion, assignment	**CLO-1**
Lesson-28	Three address code: implementatio n techniques, quadruples, triples, indirect triples and their comparison	Multimedia presentation, Whiteboard	Illustration by examples, assignment, test	**CLO-1**
Lesson-29	Run-time environments: run-time environment issues, storage organization, static vs. dynamic storage allocation	Multimedia presentation, Whiteboard	Discussion, assignment, exam	**CLO-1**
Lesson-30 & 31	Run-time environments: stack allocation, activation trees, activation of procedures, activation records, heap management	Multimedia presentation, Whiteboard	Illustration by examples, Discussion, assignment, exam	**CLO-1** **CLO-2**
Lesson-32	Code generation: primary task, issues in the design of code generation, target program, different types of target program, instruction selection, register allocation, evaluation order	Multimedia presentation, Whiteboard	Group discussion, Q/A, test	**CLO-1** **CLO-2**

Lesson-33	Code generation: code generation basic instructions, addressing mode, instruction costs	Multimedia presentation, Whiteboard	Group discussion, assignment	**CLO-1**
Lesson-34	Code generation: basic blocks, flow graphs, code generators, descriptors	Multimedia presentation, Whiteboard	Quiz, Q/A	**CLO-1**
Lesson-35 & 36	Code generation & optimization: code generation algorithm, code optimization techniques, common sub expression elimination, copy propagation, dead code elimination	Multimedia presentation, Whiteboard	Illustration with examples, Quiz, Q/A	**CLO-1**
Lesson-37	Code optimization: constant folding, practice examples	Multimedia presentation, Whiteboard	Illustration by examples, problem solving, exam	**CLO-1** **CLO-2**
Lesson-38 **Lesson-39**	**Review of previous chapters and collect feedback from students**	**Multimedia presentation, Whiteboard**	**Problem solving, open discussion, Q/A**	**CLO-2**

Part C

15. Assessment and Evaluation

l) Assessment Strategy

Class participation and attendance	10%
Class tests/Class assessment	20%
Term Final Examination (3 hours duration)	70%
Total	100%

2) Marks distribution:

a) Continuous Assessment: 30%

b) Summative: 70%

c) Make-up Procedures:

- Feedback on continuous assessment is given to the students immediately after the test.
- The minimum number of class-test/assignment are (n+1) with best n will be counted (here, n is number of credit). Based on the students' feedback additional class-test/assignment may be taken by the course teacher

Part D

16. Learning Materials

Textbook(s): "Compilers: Principles, Techniques, and Tools", Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman, (2nd Edition), Prentice hall, 2006. ISBN 0-321-48681-1

Others:

1. Kenneth C. Louden, Compiler Construction: Principles and Practice, Thomson Education, 1997

2. Y. N. Srikant and Priti Shankar, The Compiler Design Handbook: Optimizations and Machine Code Generation, Second Edition, CRC Press.